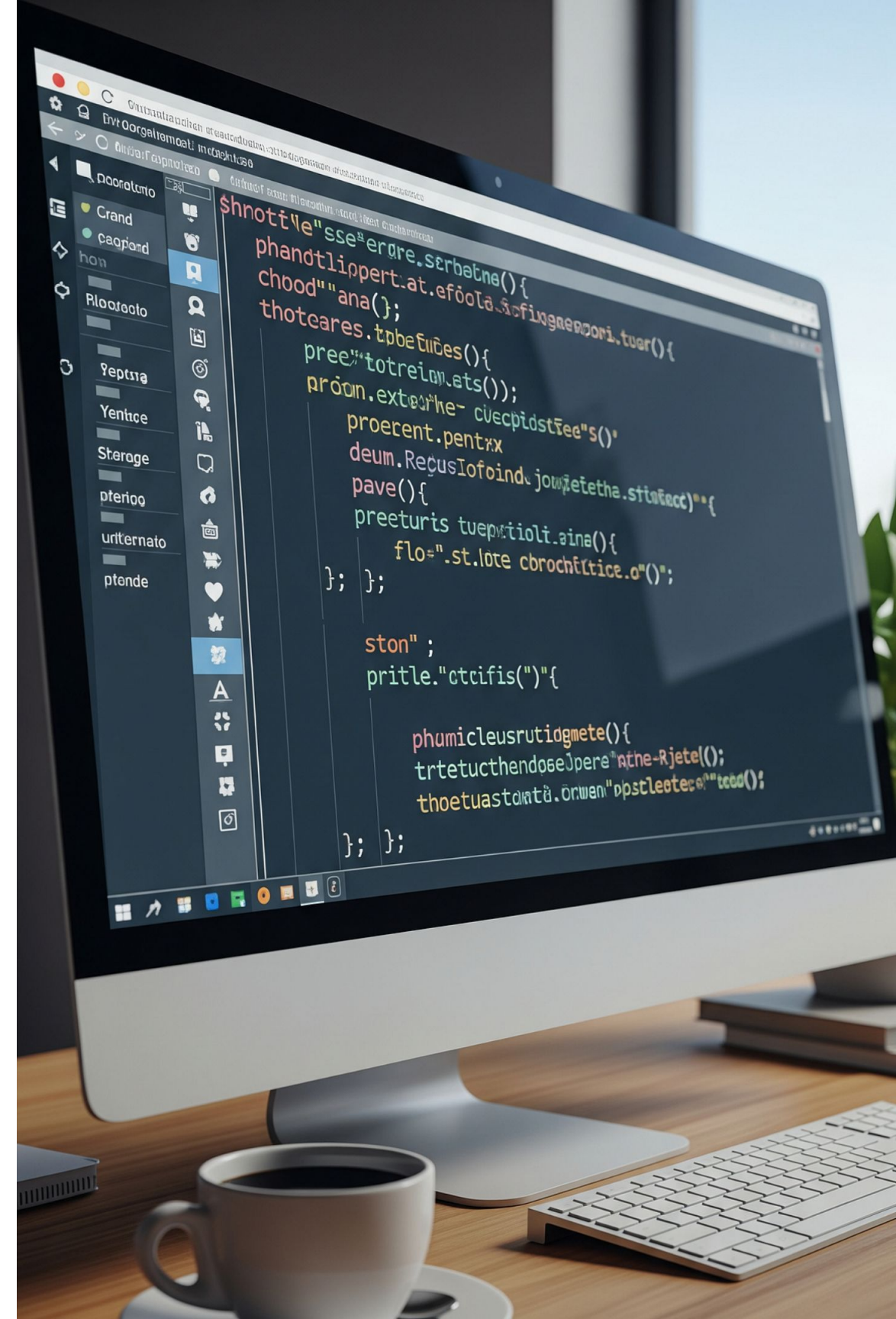


Introdução à Linguagem Java

PROGRAMAÇÃO ORIENTADA A OBJETOS

AULA 2

Bem-vindos à segunda aula da disciplina de Programação Orientada a Objetos. Nesta aula, exploraremos as bases fundamentais da linguagem Java — sua história, filosofia, arquitetura e como configurar seu ambiente de desenvolvimento.



Objetivos da Aula

Ao final desta aula, você será capaz de compreender os pilares que fazem do Java uma das linguagens mais populares do mundo e estará pronto para escrever e executar seus primeiros programas.

1

História e Evolução

Conhecer a origem do Java, seus criadores e os marcos históricos que moldaram a linguagem até hoje.

2

Características Principais

Entender os pilares técnicos: orientação a objetos, tipagem forte, garbage collection e portabilidade.

3

Funcionamento da JVM

Compreender como a Java Virtual Machine executa bytecode e garante independência de plataforma.

4

Configurar Ambiente

Instalar o JDK, configurar variáveis de ambiente e escolher uma IDE para iniciar o desenvolvimento.

Histórico do Java

1991 — Projeto Green

A Sun Microsystems inicia o projeto secreto Green, liderado por James Gosling. A linguagem original se chamava Oak, criada para dispositivos eletrônicos embarcados.

1995 — Lançamento Oficial

Java é lançado publicamente com o slogan "Write Once, Run Anywhere". A internet impulsiona sua adoção massiva em applets e aplicações web.

2006 — Open Source

A Sun libera o código-fonte do Java sob a licença GPL, dando origem ao OpenJDK e fortalecendo a comunidade de desenvolvedores.

2010 — Oracle

A Oracle Corporation adquire a Sun Microsystems por US\$ 7,4 bilhões, assumindo o desenvolvimento e a gestão do ecossistema Java.



Onde o Java é usado hoje?

Java está presente em várias áreas da tecnologia, como:

- Sistemas corporativos.
- Backend de aplicações.
- Desenvolvimento Android.
- Aplicações desktop.
- Servidores.
- Dispositivos embarcados.

Netflix: utiliza Java amplamente no backend, especialmente em microsserviços e aplicações distribuídas.

Uber: adota Java em sua base de serviços como backend, ajudando na construção de sistemas escaláveis.

Amazon: emprega Java em sistemas de grande escala, principalmente em serviços corporativos e backend.

LinkedIn: faz uso de Java em partes importantes de sua infraestrutura e de seus sistemas internos.

Filosofia Java: Write Once, Run Anywhere

O Princípio WORA

O conceito central do Java é a **portabilidade total**: um programa escrito uma única vez pode ser executado em qualquer plataforma — Windows, Linux, macOS — sem modificações no código-fonte.

Isso é possível porque o compilador Java não gera código de máquina nativo, mas sim um formato intermediário chamado **bytecode**, que é interpretado pela JVM de cada sistema operacional.

```
public class OlaMundo {  
    public static void main(String[] args) {  
        System.out.println("Olá, Java!");  
    }  
}
```

1. Esse arquivo é salvo como `OlaMundo.java`
2. O javac gera `OlaMundo.class`
3. A JVM executa esse bytecode

Características Principais da Linguagem

Orientada a Objetos

Tudo em Java é organizado em classes e objetos, exceto os tipos primitivos. Os quatro pilares: **encapsulamento**, **herança**, **polimorfismo** e **abstração** que são suportados nativamente.

Garbage Collection

O gerenciamento de memória é automático. A JVM identifica e libera objetos que não são mais referenciados, eliminando vazamentos de memória comuns em C/C++.

Tipagem Estática e Forte

O tipo de cada variável deve ser declarado em tempo de compilação. Isso reduz erros em tempo de execução e melhora a legibilidade e manutenção do código.

Sintaxe Familiar

A sintaxe do Java é fortemente inspirada em C e C++, tornando a curva de aprendizado mais suave para quem já tem experiência com linguagens de baixo nível.

JVM — Java Virtual Machine



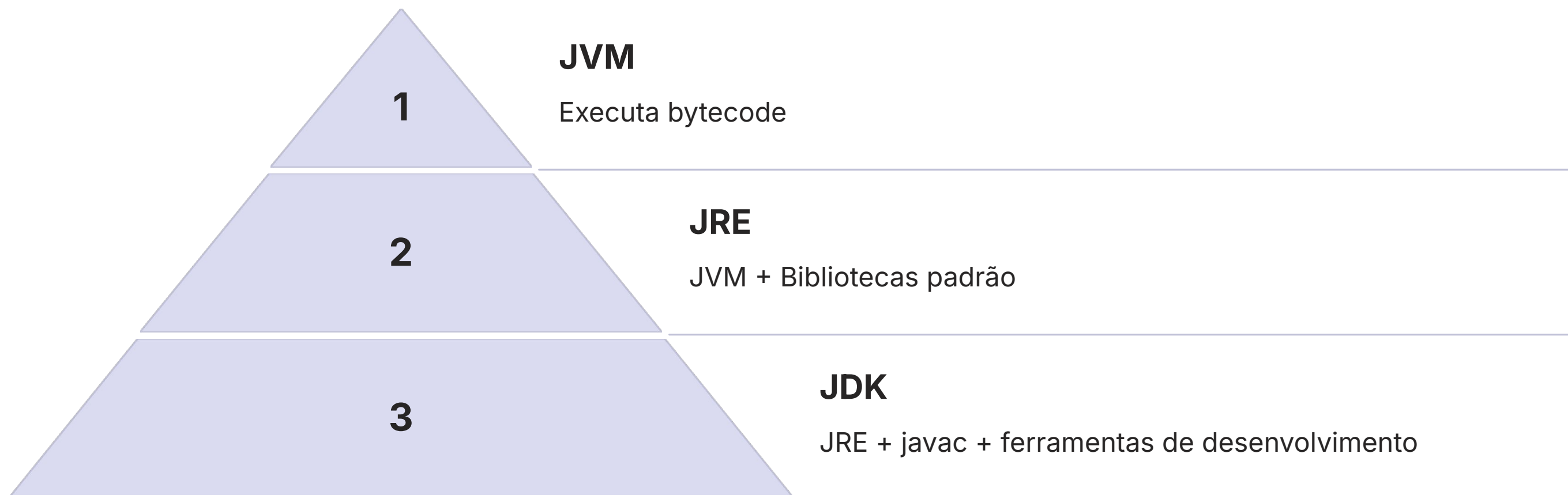
Código Fonte

Carregamento

Compilação JIT

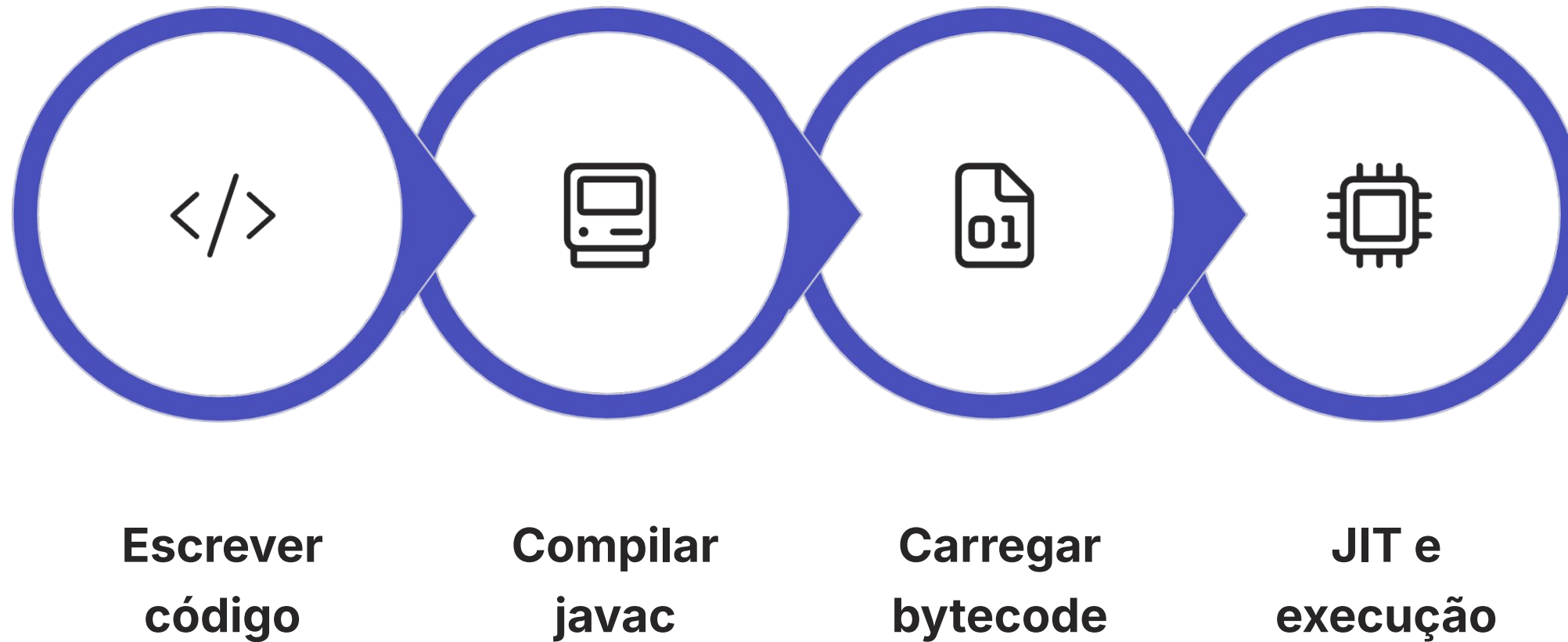
A JVM é o coração da portabilidade Java. Ela atua como uma camada de abstração entre o bytecode compilado e o hardware real. Além de executar instruções, a JVM gerencia a memória (heap e stack), aplica políticas de segurança e utiliza o compilador JIT (Just-In-Time) para otimizar o desempenho compilando trechos de bytecode frequentemente usados diretamente para código nativo.

JDK, JRE e JVM: Entendendo a Hierarquia



Entender essa hierarquia é essencial: o **JDK** (Java Development Kit) é o que você instala para desenvolver — ele inclui o compilador `javac`, o depurador e outras ferramentas. O **JRE** (Java Runtime Environment) é suficiente para quem apenas quer executar aplicações Java. A **JVM** é o núcleo que interpreta e executa o bytecode em qualquer plataforma.

Processo de Compilação e Execução



Compilação

```
javac HelloWorld.java  
# Gera: HelloWorld.class
```

Execução

```
java HelloWorld  
# JVM executa o bytecode
```

Estrutura de um Programa Java

```
package br.edu.universidade.aula;

// Declaração da classe pública
public class HelloWorld {

    // Ponto de entrada do programa
    public static void main(String[] args) {
        // Instrução de saída
        System.out.println("Olá, Mundo!");
    }
}
```



package

Organiza classes em namespaces, evitando conflitos de nomes em projetos grandes.



public class

Toda aplicação Java é centrada em classes. O nome do arquivo deve coincidir com o nome da classe pública.



main(String[] args)

Método de entrada obrigatório. A JVM inicia a execução sempre por aqui.

Tipos Primitivos em Java

Java possui **8 tipos primitivos** que não são objetos e são armazenados diretamente na memória stack, tornando operações com eles mais eficientes.

Tipo	Tamanho	Faixa de Valores	Exemplo
byte	8 bits	-128 a 127	<code>byte b = 100;</code>
int	32 bits	-2^{31} a $2^{31}-1$	<code>int n = 42;</code>
long	64 bits	-2^{63} a $2^{63}-1$	<code>long l = 99L;</code>
double	64 bits	$\pm 1.7 \times 10^{308}$	<code>double d = 3.14;</code>
char	16 bits	Unicode 0 a 65535	<code>char c = 'A';</code>
boolean	1 bit	true / false	<code>boolean ok = true;</code>

Variáveis e Constantes

Declaração de Variáveis

```
// Sintaxe: tipo nome = valor;
int idade = 20;
double salario = 3500.50;
String nome = "Maria";
boolean ativo = true;

// camelCase é a convenção
int numeroDeAlunos = 35;
String primeiroNome = "João";
```

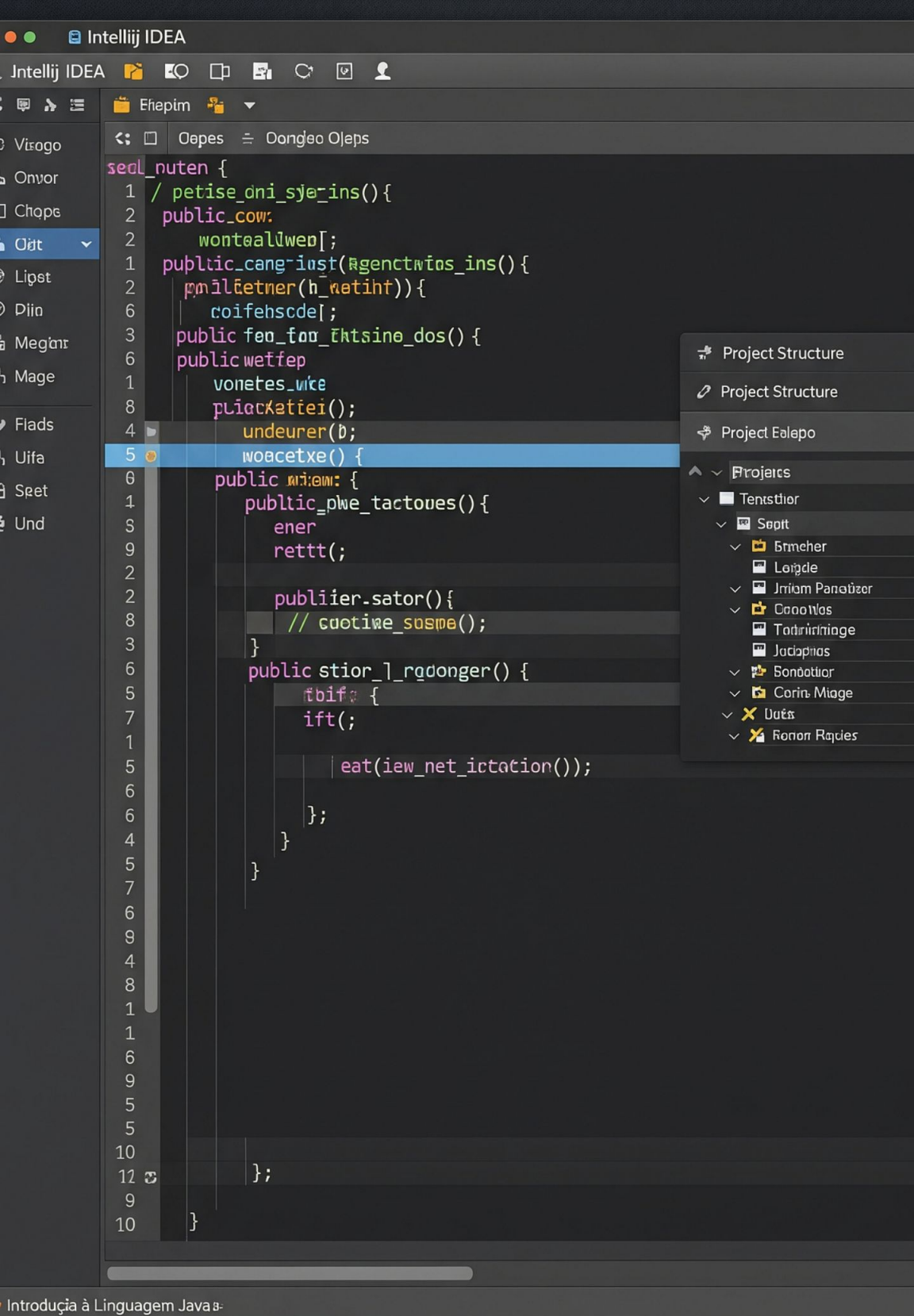
Use **camelCase** para variáveis: a primeira palavra em minúsculo e as demais com inicial maiúscula.

Declaração de Constantes

```
// Sintaxe: final tipo NOME = valor;
final double PI = 3.14159;
final int MAX_ALUNOS = 50;
final String VERSAO = "1.0.0";

// UPPER_SNAKE_CASE é a convenção
final int LIMITE_TENTATIVAS = 3;
```

Use a palavra-chave `final` para constantes e siga a convenção **UPPER_SNAKE_CASE**. O compilador impede qualquer reatribuição.



Ambiente de Desenvolvimento

IDEs Populares

IntelliJ IDEA: mais recomendada para Java

Eclipse: clássica e gratuita

VS Code: leve, com extensões Java

NetBeans: integrada com Apache

Versões do JDK

JDK 8: legado, ainda amplamente usado

JDK 11 LTS: suporte de longo prazo

JDK 17 LTS: versão recomendada

atual

JDK 21 LTS: versão mais recente LTS

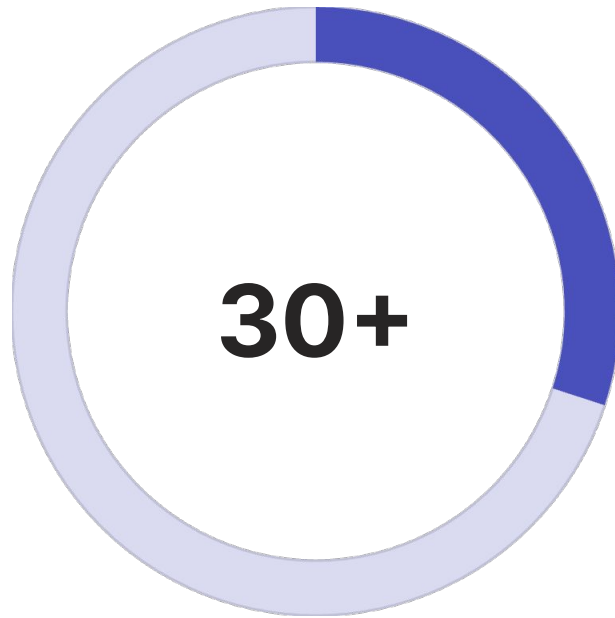
Variáveis de Ambiente

JAVA_HOME: aponta para o diretório do JDK

PATH: inclui o diretório **bin** do JDK

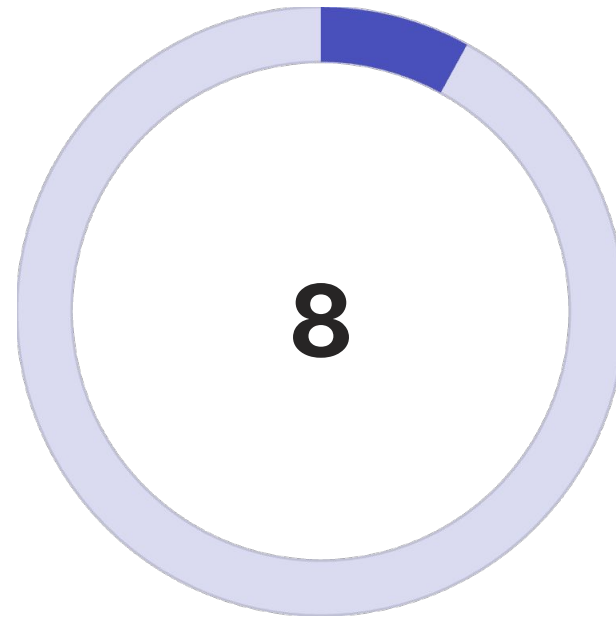
Build tools: **Maven** e **Gradle**

Recapitulando: O Ecossistema Java



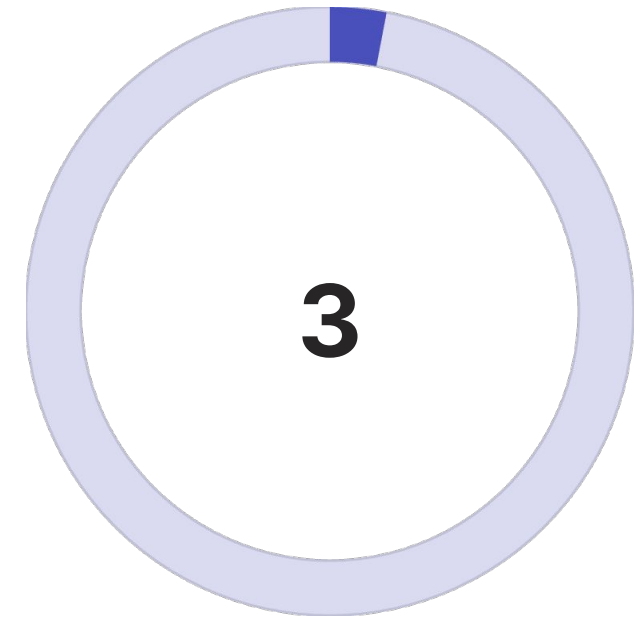
Anos de história

Do projeto Green (1991) à versão atual, Java evoluiu mantendo compatibilidade retroativa.



Tipos primitivos

byte, short, int, long, float, double, char e boolean formam a base dos dados em Java.



Camadas do ambiente

JDK, JRE e JVM formam a hierarquia que sustenta o desenvolvimento e execução de aplicações.

❏ **Conceito-chave:** O bytecode gerado pelo compilador Java é a peça central da portabilidade. Ele não é código de máquina, mas uma representação intermediária que qualquer JVM, em qualquer sistema operacional, consegue interpretar e executar com eficiência.

Próxima Aula: Orientação a Objetos

O que veremos

01

Paradigma POO

Comparação entre o paradigma procedural e o orientado a objetos — por que pensar em objetos muda tudo.

02

Conceitos Fundamentais

Classe, objeto, atributo e método — os quatro blocos de construção de qualquer programa orientado a objetos em Java.

03

Primeiros Exemplos

Modelagem de entidades do mundo real em código Java, com instanciação de objetos e acesso a atributos.

Leitura Recomendada

Capítulos 1–2 do livro base

Revise os conceitos de lógica de programação e prepare-se para absorver os fundamentos de POO.

Pratique o Hello World

Configure seu ambiente de desenvolvimento e execute o primeiro programa antes da próxima aula.

Explore a documentação

Acesse o Java SE Documentation em docs.oracle.com/javase para familiarizar-se com a referência oficial.